

Colony-Bounded Hybrid Learning for Cold-Start Fog Service Placement

Rouhollah Nabati, Abdulbaghi Ghaderzadeh

Department of Computer Engineering, *Islamic Azad University, Sanandaj Branch*, Sanandaj, Iran
rnabati@gmail.com

Department of Computer Engineering, *Islamic Azad University, Sanandaj Branch*, Sanandaj, Iran
b.ghaderzadeh80@gmail.com

Abstract

Fog service placement under burst load and membership churn is hard when the control plane itself must form from scratch. Most learning placers either assume an already organized hierarchy or score hosts over a near-global action set, so control cost and cold-start behavior are rarely isolated. We present **DynaCol-GNN**, a colony-bounded hybrid placer that keeps DynaCol’s cold-start formation, FCM handover, CRT/GRT tables, sticky L1 reuse, and cloud L3 fallback, and changes only L2 host ranking. Candidates stay inside the pruned CRT (with optional GRT escalation). Online GraphSAGE or flat-vector scorers mix with the deterministic DCBO objective through a scale-aware blend that returns to exact DCBO when the fog count is large. In iFogSim2, a 20-trial multipipe journal grid shows both hybrids reduce mean SLA versus DCBO at matched overhead on normal, burst, and churn loads, with Holm-significant paired gaps and bootstrap CIs that exclude zero. Encoder ranking between GraphSAGE and the vector ablation is mixed and non-significant. A same-plane tabular ε -greedy learner is a strong SLA control inside the colony plane; flat baselines (FogPlan, Greedy-Nearest, Static-DCBO) give absolute SLA context outside that plane. At $N=1000$ the methods stay close with matched overhead.

Keywords: fog computing, service placement, graph neural networks, hybrid learning, self-organizing systems, cold start, CRT/GRT, iFogSim2

1 Introduction

Service placement at the fog and edge must keep end-to-end delay inside SLA limits while demand, membership, and link quality move [6, 29]. When load bursts and nodes leave, placers that need a global snapshot, or that assume a ready hierarchy, either spend too much on control or fail early in the run [1, 26]. Reinforcement learning and related adapters help under changing demand [21, 25], yet many designs still open the action set to almost every host and rarely report the messaging cost of that choice [13].

DynaCol already covers the control-plane side: colonies form from a zero-colony state, an FCM is elected and handed over, and CRT/GRT tables stay bounded. DCBO is the default L2 scorer, with sticky L1 reuse and cloud L3 fallback. Missing is a learner that is *forced* to pick inside those same bounded views.

We leave formation, handover, CRT/GRT, L1, and L3 as in DynaCol and replace only L2 ranking. At each decision a GraphSAGE network scores at most the top- K CRT hosts (RTT affinity plus service-DAG context), mixes that score with inverted DCBO utility, and reduces the learning blend to zero once the fabric is large enough that exact DCBO is preferred (Fig. 1). The vector policy is the same MDP without message passing (Fig. 2).

The MDP is colony-bounded ($|A_t| \leq K$ after cold start), so the control plane is part of the learning method. L2 hybrids online scoring with DCBO and hands back to exact DCBO at large N . We compare a $HID=16$ GraphSAGE encoder (offline import plus fine-tune) with a flat vector abla-

tion and a same-plane tabular ε -greedy learner on iFogSim2 under normal/burst/churn, including a 20-trial journal grid with Holm tests, bootstrap CIs, and Cohen’s d , an $N=1000$ check, one-factor sensitivity, and named flat baselines for absolute context.

Contributions.

1. **C1 — Colony-bounded MDP.** Formal state/action/reward with $|A_t| \leq K_{\text{crt}}$ after cold start.
2. **C2 — Hybrid L2 stack.** Online scoring hybridized with DCBO plus scale-aware blend and exact large-scale DCBO delegation.
3. **C3 — Encoders and same-plane learner.** GraphSAGE with offline import and fine-tune; vector ablation; tabular DRL control on the same CRT plane.
4. **C4 — Empirical study.** Locked multipipe journal $n=20$ with Holm/CI/effect sizes, sensitivity, $N=1000$ parity, and Pareto SLA-overhead context against flat baselines.

2 Related Work

2.1 Fog and edge service placement

Surveys of fog placement cover latency, energy, and cost under fixed or centralized controllers [1, 2, 6, 23, 26, 29].

Heuristic and metaheuristic assigners, FogTorch-style QoS matching, and FogPlan Min-Cost assignment improve mapping quality on a given topology [3–5, 22, 28], but they do not form a self-organizing overlay from an empty colony state.

2.2 Learning-based offloading and placement

DQN/PPO-style and other learning placers adapt online [18, 21, 25], with applications in healthcare/fog stacks, energy-aware modules, and multi-tier containers [8, 14, 27]. Resource-management surveys still note large action spaces and the assumption of an organized fabric [7, 13, 20]. DynaCol-GNN keeps the action set inside CRT/GRT top- K and mixes learning with DCBO so cold start stays stable.

2.3 Self-organizing and hierarchical fog control

Overlay formation and manager election (including DynaCol) separate structure from placement. Hierarchical module managers and evolutionary mappers improve QoS once a hierarchy exists [9, 17], but learning is seldom constrained to the colony view as a hard limit. Here formation stays with DynaCol; learning only re-ranks inside that view.

2.4 Graph learning on infrastructure and service graphs

GraphSAGE and GNN-RL placers encode dependency graphs for scheduling and multi-objective mapping [11, 15]. CRT neighborhoods are small and RTT-structured, so local message passing fits, provided the action set stays colony-bounded. On our multipipe champion lock the GNN beats the vector ablation; we do not claim a large gap on every grid.

2.5 Relation to DynaCol

DynaCol/DCBO is the prior control plane and baseline [19]. We do not restate cold-start formation or FCM handover as new results; while that manuscript is under review, the reused algorithms are reprinted in Appendix A for self-containment. The deltas here are the colony-bounded L2 MDP, the hybrid GraphSAGE/vector learners with scale-aware blend, a same-plane tabular learner baseline, and the multipipe / journal / baseline study.

We use iFogSim / iFogSim2 for simulation [10, 16] (EdgeCloudSim is a related alternative [24]) and Holm’s procedure for multiple comparisons [12].

Table 1: Where DynaCol-GNN sits relative to common placement families.

Method family	Cold-start	Bounded view	Hybrid w/ determ. obj.	Local graph/DAG
Classical / GA / ILP	Rarely	No	Sometimes	Rarely
FogPlan / Edgeward / Static-DCBO	No/partial	No/fixed	No	No
DRL placers (DQN/PPO)	Rarely	Rarely	Rarely	Sometimes
GNN-RL placers	Rarely	Rarely	Rarely	Yes
DynaCol / DCBO	Yes	Yes	N/A	No
DynaCol-GNN	Yes	Yes	Yes	Yes

3 Background: DynaCol Control Plane

Only the reused pieces are summarized here [19]; pseudocode appears in Appendix A.

Nodes join by query/advertise until colonies appear. An FCM is elected by potency and updated with ε -greedy probes and a cooldown on handover. Each FCM keeps a CRT of member capacity, availability, RTT, and attractiveness (default top- $K_{\text{crt}}=5$); GRT summaries allow limited cross-colony search (default top- $K_{\text{gtr}}=3$) without a full host list. Placement is three-tier: sticky L1 reuse if the current host still meets SLA thresholds; DCBO L2 scoring over CRT then GRT; cloud L3 if both fail. DynaCol-GNN replaces only the L2 ranking inside CRT/GRT top- K (Fig. 1).

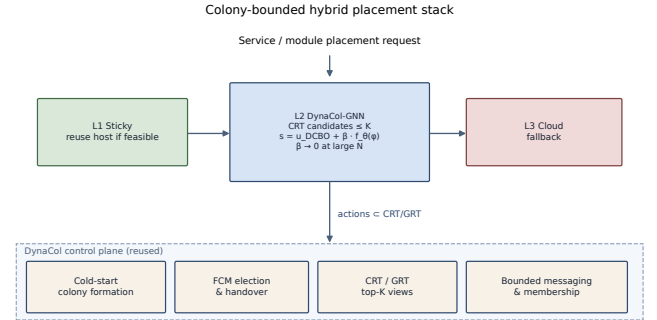


Figure 1: Placement stack on the reused DynaCol control plane: L1 sticky reuse, L2 DynaCol-GNN over CRT/GRT, L3 cloud fallback.

4 Colony-Bounded Hybrid Learning

4.1 MDP

On each FCM reconcile, state S_t is the local CRT (size $\leq K_{\text{crt}}$), optional GRT summaries, and service/DAG features. Action A_t picks a feasible CRT host or escalates to GRT/L3. The hard limit $|A_t| \leq K_{\text{crt}}$ is intentional: the control plane is part of the learning method.

Host features (dimension 8) are normalized CPU, RAM, storage, bandwidth, RTT to FCM, attractiveness, a neighbor-message channel, and producer-host affinity.

Context (dimension 8) concatenates service/DAG scalars, producer-RTT pull, fan-in, and a co-location indicator. The vector ablation zeros the co-location channel so message passing and structure-weighted context are the main encoder differences.

Reward. Let r_{attr} be the attractiveness reward used by DCBO and r_{dcbo} the same signal at the DCBO-preferred CRT host. The shaped online target is

$$R_t = \text{clip}_{[0,1]}(0.5 + 0.5(r_{\text{attr}} - r_{\text{dcbo}})). \quad (1)$$

For GNN, a graph-SLA score (producer-RTT pull, co-location under fan-in) further biases the contrastive update. Vector uses the same R_t with a weak linear TD-style update on the chosen feature vector.

4.2 Hybrid scoring

$$s(i) = u_{\text{DCBO}}(i) + \beta \cdot f_{\theta}(\phi(i)), \quad (2)$$

with u_{DCBO} the inverted DCBO objective. We take $\arg \max_i s(i)$ and allow a small ε -greedy rate while n_f is below the large-scale gate. The blend β falls with fog count (default 0.60 on multipipe at small N) and is set to zero for $n_f \geq 450$, so L2 becomes exact DCBO.

4.3 Encoders

The vector policy (`colony_rl`) concatenates host channels with service/DAG scalars and scores with a linear head (learning rate 0.05). The GNN policy (`colony_gnn`) builds an RTT-affinity graph on CRT candidates with $A_{ij} \propto 1/(1 + \text{RTT}_{ij}/20)$ and a two-layer GraphSAGE-style update (hidden width 16)

$$h' = \tanh(W_{\text{self}}h + W_{\text{neigh}}Ah), \quad (3)$$

then reads out from $\text{concat}(h, \text{context})$ (learning rate 0.04). Offline JSON weights can be loaded and fine-tuned online (Fig. 2).

A same-plane tabular baseline (`dr1`) keeps cold-start CRT/GRT formation but replaces L2 with ε -greedy selection on attractiveness ($\varepsilon=0.15$), without a neural encoder or DCBO hybridization. It is not a deep Q-network; we report it as a lightweight learning control inside the colony plane.

4.4 Online updates and offline import

After i^* is chosen, GNN applies a centered advantage update $R_t - \bar{R}$ with a soft contrastive push toward the graph-SLA-best CRT index when available; Vector applies a delta rule on the chosen features toward R_t . Attractiveness decay/update mirrors DCBO.

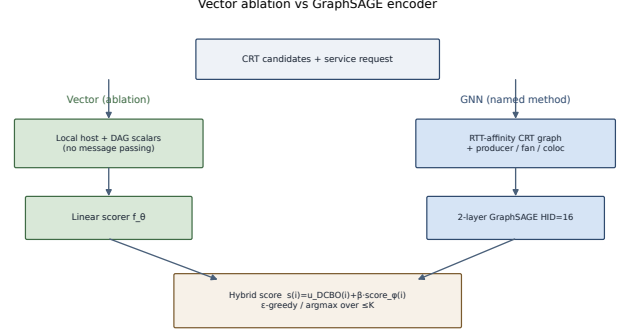


Figure 2: Vector ablation and GraphSAGE pipelines into the same hybrid score (2).

Algorithm 1 Colony-bounded hybrid reconcile (sketch)

```

1: procedure RECONCILE( $r$ , fcm, current)
2:   if  $\beta \approx 0$  then
3:     return DCBO.reconcile(...)
4:   end if
5:   if L1 sticky OK then
6:     return current
7:   end if
8:   if CRT enabled then
9:      $C \leftarrow$  top- $K$  feasible CRT entries
10:    pick  $i^*$  by hybrid score on  $\phi(C)$ ; update  $\theta$ 
11:    return host( $i^*$ )
12:   end if
13:   if GRT enabled then
14:     escalate via ranked GRT summaries
15:   end if
16:   return cloud ▷ L3
17: end procedure

```

Offline path. Exploratory `colony_gnn` runs log CRT tensors (host matrix, context, adjacency, action, reward, optional DCBO base scores). Outside the simulator we train a GIN-style encoder matching the Java layout with **reward-advantage behavioral cloning plus structure ranking** (not classic PPO): weighted NLL on the logged action, softplus ranking between structure-best/worst hosts, an optional ranking term against the inverted DCBO base vector, and a small entropy bonus. Exported JSON tensors load into the online GNN; champion and journal grids enable online fine-tuning.

4.5 Cost and algorithm

Scoring touches at most K_{crt} candidates at fixed feature/hidden width— $O(K_{\text{crt}})$ per decision with no global host enumeration. Messaging remains the CRT/GRT traffic of DynaCol.

5 Experimental Setup

Simulations run in iFogSim2 on the shared DynaCol runtime [10, 16]. We compare `dcbo`, `colony_r1` (Vector), `colony_gnn` (GNN), and same-plane tabular `drl`, plus flat `static_dcbo`, `fogplan`, and `greedy`. Edgeward was dropped on multipipe after single trials exceeded 25 minutes.

Target sizes are $N \in \{100, 300, 500, 1000\}$ under normal, burst, and churn. The surveillance DAG is the shallow camera pipeline; the multipipe DAG adds fan-out, skip, and join edges (Fig. 3) with SLA deadline scale 10. We report SLA violation %, P95 loop delay, normalized control overhead, and colony count. The tuning lock uses 5 trials; the journal $N=100$ grid uses **20** trials with paired t , Holm adjustment, bootstrap CIs, and Cohen’s d [12]; $N=1000$ uses 10 trials; flat baselines use 5 trials. One-factor sensitivity sweeps (β , deadline scale, ε) use 5 Burst trials. Locked settings: $HID=16$, $\beta=0.60$, $\varepsilon=0.02$, offline weights with fine-tune.

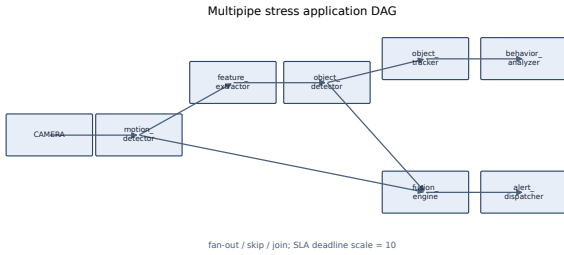


Figure 3: Multipipe application DAG used in the champion and journal grids.

6 Results

6.1 Journal multipipe grid ($n=20$)

Table 2 and Fig. 4 are the primary multipipe result. Both hybrids beat DCBO on every load at matched overhead (≈ 1770 normalized messages; ≈ 7.65 colonies). Paired bootstrap 95% CIs for GNN–DCBO and Vector–DCBO exclude zero on all three scenarios; Holm-adjusted tests clear $\alpha=0.05$ (Fig. 5). GNN vs Vector is mixed and non-significant (CIs cover zero). Same-plane tabular DRL posts the lowest SLA means with slightly lower overhead (≈ 1501) at the same colony count—a strong learning control inside the cold-start plane, not a matched-overhead peer of the hybrids.

Table 2: Multipipe journal grid: mean SLA % (20 trials, $N=100$).

N	Scenario	GNN	Vector	DCBO	Tabular-DRL
100	Normal	10.56	10.79	11.50	7.75
100	Burst	25.26	25.19	26.56	22.47
100	Churn	22.28	22.04	23.19	19.28

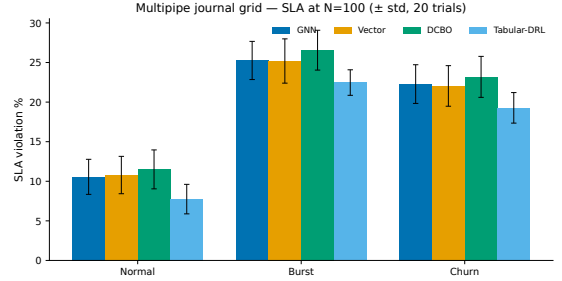


Figure 4: Multipipe journal-grid SLA at $N=100$ (20 trials).

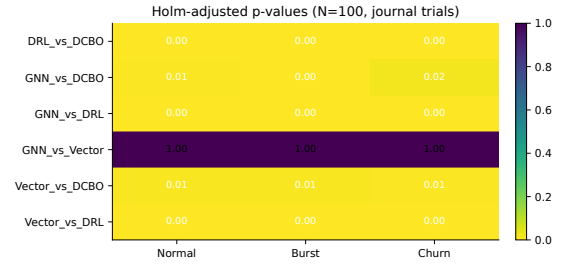


Figure 5: Holm-adjusted p -values for paired SLA tests at $N=100$ (20 trials).

6.2 Tuning lock and offline path

The 5-trial multipipe lock (Table 3, Fig. 6) fixed offline weights and hyperparameters before the journal grid; we treat it as a tuning lock, not the primary statistical claim. Without imported weights, an earlier online-only medium put Vector slightly ahead of GNN while both beat DCBO (Table 4), so the offline structure-weighted path is part of the named GraphSAGE pipeline rather than a free online GNN-over-Vector gap.

Table 3: Multipipe tuning lock: mean SLA % (5 trials, $N=100$).

N	Scenario	GNN	Vector	DCBO
100	Normal	10.64	10.98	11.48
100	Burst	24.89	25.35	26.13
100	Churn	22.31	22.43	23.21

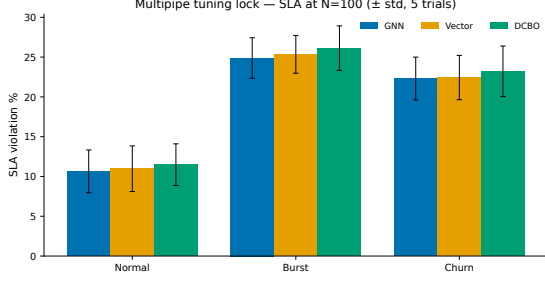


Figure 6: Multipipe tuning-lock SLA at $N=100$ (5 trials).

Table 4: Earlier online-only multipipe medium (context; pre-lock).

N	Scenario	Online-GNN	Vector	DCBO
100	Normal	10.34	10.25	11.21
100	Burst	24.39	24.09	26.42
100	Churn	21.15	21.13	23.04

6.3 Surveillance DAG

Table 5 and Fig. 7 summarize the surveillance grid. At $N=100$ the hybrid (Vector and GNN coincide on this shallow app) lowers mean SLA against DCBO at matched overhead. At $N \in \{300, 500\}$ the curves meet because the blend gate hands ranking back to DCBO (Fig. 8).

Table 5: Surveillance DAG: mean SLA violation % (5 trials).

N	Scenario	Hybrid	DCBO
100	Normal	11.51	13.31
100	Burst	14.02	14.78
100	Churn	14.58	15.37
300	all	$\approx 0.48-0.51$	$\approx 0.48-0.50$
500	all	$\approx 0.57-0.59$	$\approx 0.57-0.59$

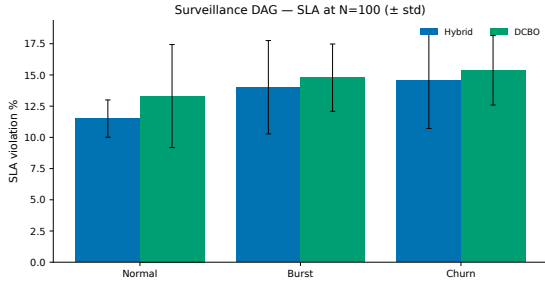


Figure 7: Surveillance SLA at $N=100$ (mean $\pm$ std, 5 trials).

6.4 Scale, overhead, and sensitivity

Fig. 8 shows surveillance mean SLA versus target size. Fig. 9 shows matched overhead for colony-plane hybrids

and DCBO. One-factor Burst sweeps (Fig. 10) place $\beta=0.60$ and $\varepsilon=0.02$ in the interior of mild SLA variation; deadline scale shifts absolute SLA sharply (as expected) while hybrid-over-DCBO direction holds at scales 8 and 12. The gate uses the live fog count ($n_f \geq 450 \Rightarrow$ exact DCBO).

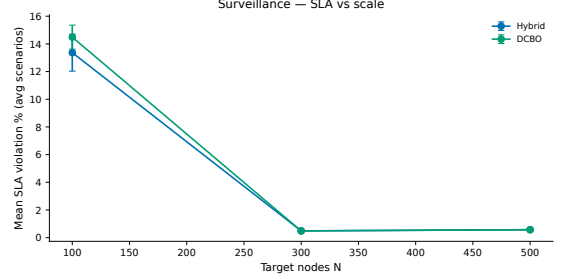


Figure 8: Surveillance mean SLA versus target size N (averaged over scenarios).

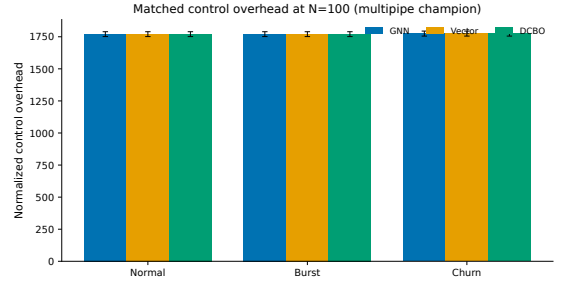


Figure 9: Normalized control overhead for colony-plane methods ($N=100$). GNN, Vector, and DCBO remain matched.

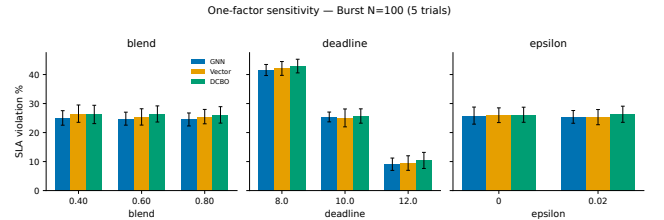


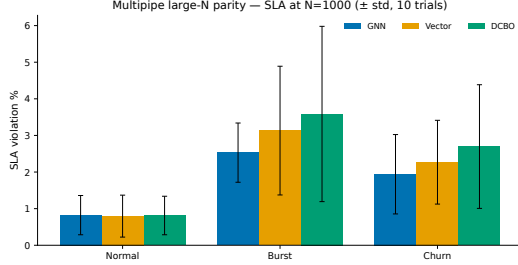
Figure 10: One-factor sensitivity on Burst $N=100$ (5 trials): blend, deadline scale, and ε .

6.5 Large- N check

At $N=1000$ (Table 6, Fig. 11) absolute SLA is low and overhead matches ($\approx 2112-2140$). Burst/churn means still favor GNN; Holm tests after correction remain mostly non-significant, consistent with near-parity under the large- N blend gate.

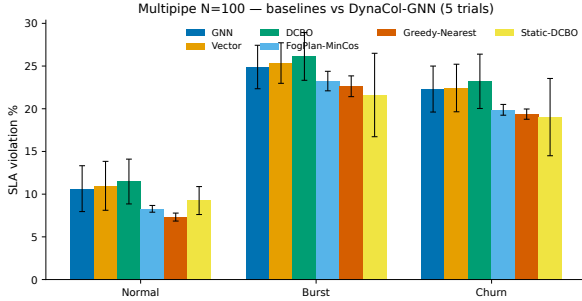
Table 6: Multipipe at $N=1000$: mean SLA % (10 trials).

Scenario	GNN	Vector	DCBO	Overhead
Normal	0.82	0.80	0.81	2112
Burst	2.53	3.13	3.59	2112
Churn	1.94	2.27	2.70	2140

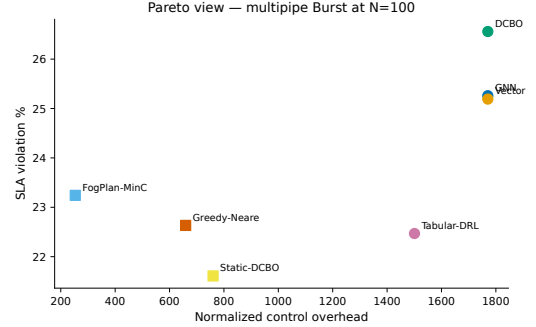
**Figure 11:** Multipipe SLA at $N=1000$ (mean $\pm$ std, 10 trials).

6.6 Named baselines and Pareto view

FogPlan, Greedy-Nearest, and Static-DCBO can post lower multipipe SLA than the colony hybrids (Table 7, Fig. 12), but overhead and colony counts are not comparable. Fig. 13 plots SLA against normalized overhead on Burst: flat methods sit at lower cost; colony-plane methods cluster together; tabular DRL sits between them on cost while leading SLA inside the cold-start plane. The claim we defend is therefore *within-plane* hybrid improvement over DCBO at matched hybrid/DCBO overhead.

**Figure 12:** Flat baselines versus colony-bounded methods on multipipe $N=100$ (5 trials).**Table 7:** Multipipe $N=100$: flat baselines (5 trials) beside the tuning-lock colony means. Overhead is not matched across planes.

Scenario	FogPlan	Greedy	Static	GNN	Vector	DCBO
Normal	8.28	7.32	9.25	10.64	10.98	11.48
Burst	23.24	22.63	21.61	24.89	25.35	26.13
Churn	19.88	19.37	19.02	22.31	22.43	23.21
Overhead	253	659	760	≈ 1769	≈ 1769	≈ 1769
Colonies	1	1	19	≈ 7.6	≈ 7.6	≈ 7.6

**Figure 13:** Pareto view: SLA vs normalized control overhead (Burst, $N=100$).

7 Discussion

Bounding $|A_t|$ ties learning to the same CRT/GRT traffic DynaCol already exchanges. Inside that plane, hybrid L2 improves $N=100$ multipipe SLA versus DCBO with Holm-significant paired evidence at $n=20$ (Table 2).

The title emphasizes *colony-bounded hybrid learning* rather than a universal GNN-over-Vector gap: encoder ranking is mixed and non-significant on the journal grid, while GraphSAGE remains the named structured encoder with an offline BC+ranking import path. Tabular DRL shows that a lightweight learner on the same CRT can lead absolute SLA; the hybrid’s distinctive properties are DCBO anchoring, scale-aware exact fallback, and structure-aware offline initialization—not raw dominance over every same-plane learner.

Flat baselines in Table 7 and Fig. 13 are useful for absolute SLA, not for a same-plane ranking. We do not claim new cold-start formation, new FCM election, or testbed validation. Limits include simulation-only evidence, calibrated multipipe deadlines (Fig. 10), logged (not fully on-policy) offline data, and the omission of Edgeward on multipipe for runtime reasons.

8 Conclusion

DynaCol-GNN keeps fog placement inside cold-start colony views: L2 mixes online scoring with DCBO, actions stay in CRT/GRT top- K , and large fabrics fall back to exact

DCBO. On iFogSim2 the hybrid lowers SLA at $N=100$ versus DCBO at matched overhead on surveillance and multipipe workloads, with Holm-significant multipipe gaps at $n=20$, and stays close at $N=1000$. GraphSAGE and the vector ablation are statistically tied as encoders; a same-plane tabular learner provides a strong SLA control. Flat baselines remind the reader that absolute SLA and control-plane cost can trade off outside the colony plane.

Next steps include CRT-ablated learning, stronger on-policy offline training, deeper same-plane neural baselines, and testbed runs.

Declarations

Funding. The authors declare that no funds, grants, or other support were received during the preparation of this manuscript.

Competing interests. The authors have no relevant financial or non-financial interests to disclose.

Author contributions. R.N. designed the method, implemented the experiments, and drafted the manuscript. A.G. supervised the research and revised the manuscript. Both authors read and approved the final manuscript.

Data availability. Simulation configs, trial CSVs, summary tables, and figure scripts are included in the accompanying dynacol-gnn artifact directory (`configs/`, `evaluation/results/`, `evaluation/`).

Code availability. Source code for the colony-bounded placers and evaluation harness is provided with the artifact (Java iFogSim2 extension under `ifogsim2/` and Python tools under `dynacol-gnn/`).

A Reused DynaCol algorithms

For self-containment while the prior DynaCol manuscript is under review [19], we reprint the cold-start, handover, and DCBO L1–L3 skeletons that this paper leaves unchanged. These algorithms are *not* claimed as contributions here.

B Hyperparameters

References

- [1] F. Ait Salaht, F. Desprez, and A. Lebre. An overview of service placement problem in fog and edge computing. *ACM Computing Surveys*, 53(3):65:1–65:35, 2020.
- [2] H. K. Apat, V. Goswami, B. Sahoo, R. K. Barik, and M. J. Saikia. Fog service placement optimization: A

Algorithm 2 Cold-start join (Create_And_Add_FogNode)

Require: join event for node FID ; timers W_t , T_{adv} ; radius $MaxColonySize$

- 1: state $\leftarrow$ Unassigned
- 2: broadcast $QueryColony(FID)$; start timer T
- 3: **if** no response and $T \geq W_t$ **then**
- 4: self-elect FCM; init CRT/GRT;
 AdvertisePeriodically(T_{adv})
- 5: **else**
- 6: $resp \leftarrow$ best JoinScore response
- 7: **if** $L(FID, resp.FCM) < MaxColonySize$ **then**
- 8: JoinOrNegotiate($resp$) ▷ Alg. 3
- 9: **end if**
- 10: **end if**

Algorithm 3 JoinOrNegotiate (potency-based FCM handover)

Require: response $resp$ with colony id, FCM, CRT

- 1: **if** self is FCM-candidate and $Potency(self) > Potency(resp.FCM) + \epsilon$ and cooldown elapsed **then**
- 2: negotiate ManagerTakeoverRequest; on accept promote self and merge CRT
- 3: **else**
- 4: join $resp.ColonyID$ under $resp.FCM$; FCM updates CRT/GRT
- 5: **end if**

Algorithm 4 DCBO_PlaceService (L1–L3)

Require: service s , current host h_{cur} , tables CRT, GRT

- 1: **if** h_{cur} feasible and SLA-safe **then**
- 2: **return** h_{cur} ▷ L1 sticky
- 3: **end if**
- 4: $Cand \leftarrow$ top- K_{crt} feasible CRT hosts by $J(s, \cdot)$
- 5: **if** $Cand \neq \emptyset$ **then**
- 6: **return** arg min $Cand$ ▷ L2 local
- 7: **end if**
- 8: **for** all col in top- K_{grt} (GRT) **do**
- 9: **if** DelegatePlace(s , col.FCM) succeeds **then**
- 10: **return** delegated host ▷ L2 neighbor
- 11: **end if**
- 12: **end for**
- 13: **return** cloud by latency/cost ▷ L3

survey of state-of-the-art strategies and techniques. *Computers*, 14(3):99, 2025.

- [3] M. Azimzadeh, A. Rezaee, S. J. Jassbi, and M. Esnaashari. Placement of IoT services in fog environment based on complex network features: A genetic-based approach. *Cluster Computing*, 25(5):3423–3445, 2022.
- [4] A. R. Benamer, N. B. Hadj-Alouane, K. Boussetta, and A. B. Hadj-Alouane. A genetic algorithm-

Table 8: Locked-grid hyperparameters.

Parameter	Default / note
CRT top- K_{crt} / GRT top- K_{grt}	5 / 3
ε (hybrid)	0.02 on multipipe (0 at large N)
ε (tabular DRL)	0.15
Vector / GNN LR	0.05 / 0.04
GNN hidden	16, 2 layers, tanh
Host / context dims	8 / 8
Blend (multipipe)	0.60
Multipipe SLA scale	10.0
Large- N gate	$n_f \geq 450 \rightarrow$ exact DCBO
Offline objective	reward-advantage BC + structure ranking
Trials	5 (tuning/baselines); 20 (journal $N=100$); 10 ($N=1000$)

- based approach for placement in the fog of latency-sensitive multiplayer game servers. *Cluster Computing*, 27:11249–11275, 2024.
- [5] A. Brogi and S. Forti. Qos-aware deployment of IoT applications through the fog. *IEEE Internet of Things Journal*, 4(5):1185–1192, 2017.
- [6] M. Chiang and T. Zhang. Fog and iot: An overview of research opportunities. *IEEE Internet of Things Journal*, 3(6):854–864, 2016.
- [7] B. Costa, J. Bachiega Jr., L. R. de Carvalho, and A. P. F. Araujo. Orchestration in fog computing: A comprehensive survey. *ACM Computing Surveys*, 55(2), 2022.
- [8] J. Dogani, A. Yazdanpanah, A. Zare, and F. Khunjush. A two-tier multi-objective service placement in container-based fog-cloud computing platforms. *Cluster Computing*, 27(4):4491–4514, 2024.
- [9] C. Guerrero, I. Lera, and C. Juiz. Evaluation and efficiency comparison of evolutionary algorithms for service placement optimization in fog architectures. *Future Generation Computer Systems*, 97:131–144, 2019.
- [10] H. Gupta, A. Vahid Dastjerdi, S. K. Ghosh, and R. Buyya. ifogsim: A toolkit for modeling and simulation of resource management techniques in the internet of things, edge and fog computing environments. In *Proc. of IEEE/ACM International Conference on Utility and Cloud Computing (UCC)*, pages 47–56, 2017.
- [11] W. L. Hamilton, R. Ying, and J. Leskovec. Inductive representation learning on large graphs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [12] S. Holm. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70, 1979.
- [13] C.-H. Hong and B. Varghese. Resource management in fog/edge computing: A survey on architectures, infrastructure, and algorithms. *ACM Computing Surveys*, 52(5), 2019.
- [14] H. S. Hossam, H. Abdel-Galil, and M. Belal. An energy-aware module placement strategy in fog-based healthcare monitoring systems. *Cluster Computing*, 27:7351–7372, 2024.
- [15] I. Lera and C. Guerrero. Multi-objective application placement in fog computing using graph neural network-based reinforcement learning. *The Journal of Supercomputing*, 80:27073–27094, 2024.
- [16] R. Mahmud, S. Pallegatta, M. Goudarzi, and R. Buyya. ifogsim2: An extended ifogsim simulator for mobility, clustering, and microservice management in edge and fog computing environments. *Journal of Systems and Software*, 190:111306, 2022.
- [17] R. Mahmud, K. Ramamohanarao, and R. Buyya. Latency-aware application module management for fog computing environments. *ACM Transactions on Internet Technology*, 19(1):1–21, 2018.
- [18] V. Mnih et al. Human-level control through deep reinforcement learning. *Nature*, 518:529–533, 2015.
- [19] R. Nabati and A. Ghaderzadeh. DynaCol: Self-organizing fog colonies with cold-start formation and bounded resource tables. Submitted to *Cluster Computing* (under review), 2026.
- [20] S. Pallegatta, V. Kostakos, and R. Buyya. Placement of microservices-based iot applications in fog computing: A taxonomy and future directions. *ACM Computing Surveys*, 55(14s):321:1–321:43, 2023.
- [21] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. In *arXiv preprint arXiv:1707.06347*, 2017.

- [22] O. Skarlat, M. Nardelli, S. Schulte, and S. Dustdar. Towards qos-aware fog service placement. In *Proc. IEEE 1st International Conference on Fog and Edge Computing (ICFEC)*, pages 89–96, 2017.
- [23] S. Smolka et al. Evaluation of fog application placement algorithms: A survey. *Computing*, 104:2373–2415, 2022.
- [24] C. Sonmez, A. Ozgovde, and C. Ersoy. Edge-cloudsim: An environment for performance evaluation of edge computing systems. *Transactions on Emerging Telecommunications Technologies*, 29(11):e3493, 2018.
- [25] T. Supraja, P. Chawla, and S. S. Gill. AI-Driven Service Placement in Fog and Edge Computing Environments: A Systematic Literature Review, Taxonomy and Future Directions. *Cluster Computing*, 28(16):1070, 2025.
- [26] I. Taleb, J.-L. Guillaume, and B. Duthil. A survey on services placement algorithms in integrated cloud-fog/edge computing. *ACM Computing Surveys*, 57(11), 2025.
- [27] S. Tuli, N. Basumatary, S. S. Gill, M. Kahani, R. C. Arya, G. S. Wander, and R. Buyya. HealthFog: An ensemble deep learning based smart healthcare system for automatic diagnosis of heart diseases in integrated IoT and fog computing environments. *Future Generation Computer Systems*, 104:187–200, 2020.
- [28] A. Yousefpour, C. Fung, T. Nguyen, K. Kadiyala, F. Jalali, et al. Fogplan: A lightweight qos-aware dynamic fog service provisioning framework. *IEEE Internet of Things Journal*, 6(3):5080–5096, 2019.
- [29] A. Yousefpour, C. Fung, T. Nguyen, K. Kadiyala, F. Jalali, A. Niakanlahiji, J. Kong, and J. P. Jue. All one needs to know about fog computing and related edge computing paradigms: A complete survey. *Journal of Systems Architecture*, 98:289–330, 2019.